

File Stream Handling in C++ (ifstream, ofstream, fstream)

[DEF] Definition to Remember

File handling in C++ is managed via stream classes (<fstream>: ifstream for reading, ofstream for writing, fstream for both), providing object-oriented persistent storage.

1. Concept Overview & Fundamentals

File Stream Handling in C++ (ifstream, ofstream, fstream) provides modern high-performance capabilities combining low-level hardware control with high-level abstractions, object-oriented paradigms, and Standard Template Library (STL) utilities.

In modern C++ (C++11 through C++20):

2. Technical Architecture & Memory Mechanics

C++ Runtime Execution Architecture



Modern C++ Feature Comparison

Modern C++ Paradigm	Legacy Equivalent	Engineering Advantage	Performance Impact
<code>**std::cin` / `std::cout`**</code>	<code>`scanf` / `printf`</code>	Type-safe, extensible for custom class	Buffered I/O (optimizable via <code>`ios::sy</code>
<code>**References (`Type&`)**</code>	Pointers (<code>`Type*`</code>)	Non-nullable, clean syntax without der	Direct alias; zero runtime overhead
<code>**std::vector<T>`**</code>	Raw Dynamic Array (<code>`malloc`</code>)	Automatic resizing, bounds safety, mem	Amortized <code>**O(1)**</code> push_back, contiguo
<code>**`nullptr`**</code>	<code>`NULL` / `0`</code>	Strongly typed pointer literal prevent	Zero overhead

3. Core Syntax & Implementation Patterns

Modern C++ Template Structure

* Quantitative Finance & HFT: High-frequency trading platforms use modern C++ for microsecond-level execution.

```

/*
 * Que
 * Ope
 */

/*
 * ==
 * COM
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ==
 */

```

Page 2/9 | Created by Suraj (B.Tech DS) | GATE 2028

```
// Use
using

//
int ma
of

//
if

}

st
//
cc
//
ge

ld
//
cc

ld
//
re
}
```

Problem 2: check file open

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Att
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 * ==
 */
```

Problem Description & Objective:

Implement

```

/*
 * Que
 * Cop
 */

/*
 * ==
 * COM
 * ==
 * 1.
 *
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 * ==

```

```
*/  
  
#inclu  
#inclu  
  
// Use  
using  
  
//  
int ma  
if  
of  
  
//  
if  
  
  
}  
  
ch  
in  
//  
wh  
  
}  
  
//  
co  
  
sn  
de  
  
//  
re  
  
}
```

Problem 4: read write file

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Wri  
 */  
  
/*  
 * ===  
 * CON  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 *  
 */
```

```
* 3.
*
*
* 4.
* *
*
*
*
* ==
*/

#include
#include
#include

// Use
using

//
int ma
st

of
ou
ou
ou

if
st
//
co
//
wh

}
in

//
re

}
```

Problem 5: sum from file

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Rea
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 * 2.
 *
```

```
*
* 3.
*
*
* 4.
* *
*
*
*
* ===
*/

#include
#include

// Use
using

//
int ma
st

of
ou
ou

if
in

//
wh

}
in

//
co
//
co

//
re
}
```

Problem 6: write lines

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
* Que
* Wri
*/

/*
* ===
* CON
* ===
* 1.
*
*/
```

```
*
* 2.
*
*
* 3.
*
*
* 4
* *
*
*
*
* ==
*/

#include
#include
#include

// Use
using

//
int ma
of

//
if

}

in
//
co
//
ci
ci

//
co
//
fo

}

ou
//
co

//
re

}
```

6. Exam Scoring Points & Memory Revision

[MUST-WRITE] Must-Write Points

1. `std::ifstream` reads files; `std::ofstream` writes files; `std::fstream` handles dual-mode I/O.
2. File opening: Open via constructor or `file.open("path", ios::mode)`.
3. Stream state validation: `file.is_open()` and `while(file >> data)` safely detect EOF.
4. Closing files: RAII destructors automatically close streams, but explicit `file.close()` is best practice.
5. Binary vs Text: `ios::binary` flag prevents OS-specific newline translations in binary mode.

[RECALL] Quick Recall

`ifstream / ofstream` -> `is_open()` Check -> Stream `<< / >>` -> `getline` Loop -> `file.close()`